ID-AIDemo 架构升级指南

> 前提：不修改 idbackend / idfrontend / idfrontend-admin，只改 ID-AIDemo 工程内部。
> 每个阶段独立可验证，改完一个阶段就能跑，别跳步。

当前项目结构（升级前）

```
src/
├── agent/
|   ├── mainGraph.ts          ← 图定义 + HTTP适配(invokeAgent/streamAgent) 混在一起
|   ├── state.ts
|   └── subGraphs/
|       ├── applyGraph.ts     ← 节点里直接 new ChatOpenAI + 写死 prompt
|       └── consultGraph.ts   ← 同上
├── db/init.ts
├── routes/components/        ← 每个路由文件自己创建 multer、UPLOAD_DIR
|   ├── agent.ts
|   ├── analyze.ts
|   ├── config.ts
|   └── knowledge.ts
├── services/
|   ├── aiConfig.ts           ← DB读写 + 缓存，结构重
|   └── knowledgeManager.ts   ← MemoryVectorStore，重启丢数据
├── types/scoreTemplate.ts
└── index.ts
```

当前存在的问题

1. **mainGraph.ts 职责过重**：图定义 + HTTP 适配（invokeAgent/streamAgent/extractResult/checkInterrupt）全在一个 248 行的文件里
2. **Prompt 散落各处**：classifyNode 里 40 行 prompt、applyGraph 里写死 prompt、consultGraph 里又有一套，改一个要翻三个文件
3. **LLM 创建重复**：`new ChatOpenAI({ apiKey: getApiKey(), ... })` 在 mainGraph、applyGraph、consultGraph、analyze.ts 里写了 4 遍
4. **multer/UPLOAD_DIR 重复**：agent.ts、analyze.ts、knowledge.ts 三个路由文件各自创建一遍
5. **知识库/对话不持久**：MemoryVectorStore 重启丢数据，MemorySaver 重启丢会话
6. **aiConfig 过于复杂**：为了"在线改配置"引入了整套 DB 读写 + 缓存 + TTL

升级后目标结构

```
src/
├── agent/
|   ├── prompts.ts            ← 【新】所有 prompt 模板集中管理
|   ├── graphs/
|   |   ├── mainGraph.ts      ← 【瘦身】只有纯图定义
```

```
|   |   ├── applyGraph.ts
|   |   └── consultGraph.ts
|   └── state.ts
├── db/init.ts
├── routes/
|   ├── index.ts
|   └── components/
|       ├── agent.ts          ← 【瘦身】只做参数解析 + 调 agentService
|       ├── analyze.ts
|       ├── config.ts
|       └── knowledge.ts
├── services/
|   ├── agentService.ts       ← 【新】从 mainGraph 拆出的 HTTP 适配层
|   ├── llmService.ts         ← 【新】统一的 LLM/Embedding 工厂
|   ├── knowledgeService.ts   ← 【重命名+升级】持久化向量存储
|   └── aiConfig.ts           ← 【简化】改为 .env 为主
├── shared/
|   └── upload.ts             ← 【新】公共 multer 配置
├── types/scoreTemplate.ts
└── index.ts
```

阶段 1：集中管理 Prompt

> 目标：所有 prompt 文本从图节点中抽出，统一放到 `agent/prompts.ts`

1.1 新建 `src/agent/prompts.ts`

把所有散落在各个节点的 prompt 文本集中到一个文件。用函数返回，这样可以接受动态参数：

```
// src/agent/prompts.ts

/** classify 节点的意图分类 prompt */
export function classifyPrompt(allUserText: string): string {
  return `分析以下用户的多轮输入，判断意图并提取信息：

【分类与校验规则】：
1. 如果用户是单纯询问政策或了解相关信息（如"挑战杯能加多少分"），意图记为 consult。
2. 如果用户表达了"想申请加分"或正在提交比赛凭证，必须严格校验其提供的信息是否完整！
   完整材料必须同时包含以下4个关键要素：【赛事名称】、【奖项等级】、【时间】、【申请人角色】。
   * 特别注意：赛事名称允许使用简称（如"挑战杯"、"互联网+"、"国创"等），只要提到了即可，不要强求全称。
3. 如果是申请意图，但上述4个要素有任何缺失，意图必须记为 insufficient，并在 missing 数组中准确列出还缺少的要素名称。
4. 如果是申请意图，且上述4个要素全部齐备，意图记为 apply，并将所有用于申请的原始材料文本合并提取到 documentText 中。

【用户输入历史】：
${allUserText}`
}

/** consult 子图的系统角色 prompt */
export function consultSystemPrompt(systemRole: string, context: string): string {
```

```
  return `${systemRole}\n\n【知识库检索结果】\n${context}`
}

/** apply 子图的材料分析 prompt */
export const ANALYZE_SYSTEM = '你是厦门大学信息学院推免加分审核专家。分析证明材料，判断可以申请哪些加分项。如果无匹配返回空数组。'

export function analyzeUserPrompt(
  documentText: string,
  templatesJson: string,
  policyContext: string
): string {
  return `学生上传了以下证明材料：
---
${documentText.slice(0, 2000)}
---

可申请的加分模板列表：
${templatesJson}

相关加分政策参考：
${policyContext}

请分析证明材料，判断学生可以申请哪些加分项。`
}

/** apply 子图的结果汇总（无匹配时） */
export const NO_MATCH_REPLY = '根据您提供的材料，暂未匹配到符合条件的加分项。请确认材料内容是否完整，或补充更多信息。'

/** 申请信息不完整时的追问模板 */
export function askForMorePrompt(missingFields: string[]): string {
  return `申请材料不完整，还缺：${missingFields.join('、')}。请补充：`
}
```

1.2 改造各节点引用 prompts

以 `mainGraph.ts` 的 classifyNode 为例，改造前后对比：

改造前（直接写在节点里）：

```
const messages = [new HumanMessage(`
    分析以下用户的多轮输入，判断意图并提取信息：
    ...40行prompt...
    ${allUserText}
`)]
```

改造后：

```
import { classifyPrompt } from '../prompts.js'

// classifyNode 中:
const messages = [new HumanMessage(classifyPrompt(allUserText))]
```

同理改造 `consultGraph.ts`、`applyGraph.ts`、`analyze.ts` 中的所有 prompt。

1.3 验证

改完后 `npm run dev`，用前端发几条消息测试：

- 发 "你是谁" → 走 consult 路径，应正常回复
- 发 "我参加了挑战杯，省级一等奖" → 走 insufficient → 追问缺失字段
- 确认 prompt 行为和之前完全一致

阶段 2：统一 LLM 工厂

> 目标：消除 `new ChatOpenAI({ apiKey: getApiKey(), ... })` 的 4 处重复

2.1 新建 `src/services/llmService.ts`

```
// src/services/llmService.ts
import { ChatOpenAI } from '@langchain/openai'
import { OpenAIEmbeddings } from '@langchain/openai'
import { getApiKey, getBaseUrl, getChatModel, getEmbeddingModel } from './aiConfig.js'

/** 创建聊天模型（各节点共享配置，只需传 temperature） */
export function createChatModel(temperature = 0.3) {
  return new ChatOpenAI({
    apiKey: getApiKey(),
    configuration: { baseURL: getBaseUrl() },
    modelName: getChatModel(),
    temperature,
  })
}

/** 创建 embedding 模型 */
export function createEmbeddings() {
  return new OpenAIEmbeddings({
    openAIApiKey: getApiKey(),
    configuration: { baseURL: getBaseUrl() },
    modelName: getEmbeddingModel(),
    batchSize: 6,
    maxRetries: 3,
  })
}
```

2.2 全局替换

- `mainGraph.ts` classifyNode：`createChatModel(0).withStructuredOutput(...)`
- `consultGraph.ts` answerNode：`createChatModel(0.3)`
- `applyGraph.ts` analyzeAndMatchNode：`createChatModel(0.1).withStructuredOutput(...)`
- `analyze.ts` createModel：删掉，改用 `import { createChatModel } from '../../services/llmService.js'`
- `knowledgeManager.ts` createEmbeddings：删掉，改用 `import { createEmbeddings } from './llmService.js'`

2.3 验证

同阶段 1，确认所有路径（consult/apply/analyze/knowledge上传）都能正常工作。

阶段 3：公共 multer + 拆分 agentService

> 目标：
>
> - 消除 3 个路由文件中重复的 multer/UPLOAD_DIR
> - 把 mainGraph.ts 中的 HTTP 适配逻辑拆到 agentService

3.1 新建 `src/shared/upload.ts`

```
// src/shared/upload.ts
import multer from 'multer'
import path from 'path'
import { fileURLToPath } from 'url'
import fs from 'fs'

const __dirname = path.dirname(fileURLToPath(import.meta.url))
export const UPLOAD_DIR = path.resolve(__dirname, '../../uploads')
fs.mkdirSync(UPLOAD_DIR, { recursive: true })

/** 通用上传（任意文件类型，10MB 限制） */
export const upload = multer({
  dest: UPLOAD_DIR,
  limits: { fileSize: 10 * 1024 * 1024 },
})

/** 知识库上传（保留原始文件名，20MB 限制） */
export const knowledgeUpload = multer({
  storage: multer.diskStorage({
    destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
    filename: (_req, file, cb) => {
      const decoded = Buffer.from(file.originalname, 'latin1').toString('utf8')
      cb(null, decoded)
    },
  }),
  limits: { fileSize: 20 * 1024 * 1024 },
```

```
})
```

然后 `agent.ts`、`analyze.ts`、`knowledge.ts` 都从这里 import，删掉各自的 multer 配置。

3.2 新建 `src/services/agentService.ts`

把 `mainGraph.ts` 中 100-247 行（`AgentInput`、`extractResult`、`checkInterrupt`、`invokeAgent`、`streamAgent` 等）全部搬到这里：

```
// src/services/agentService.ts
import { HumanMessage } from '@langchain/core/messages'
import { Command } from '@langchain/langgraph'
import { getCompiledGraph } from '../agent/graphs/mainGraph.js'
import type { ScoreTemplate } from '../types/scoreTemplate.js'

export interface AgentInput {
  userInput: string
  documentText?: string
  templates?: ScoreTemplate[]
  sessionId: string
}

const app = getCompiledGraph()  // mainGraph 只 export 编译好的图

function extractResult(state: any) { /* 原逻辑不变 */ }
async function checkInterrupt(config: any) { /* 原逻辑不变 */ }

export async function invokeAgent(input: AgentInput) { /* 原逻辑不变 */ }
export async function resumeAgent(sessionId: string, supplement: string) { /* 原逻辑不变 */ }
export async function* streamAgent(input: AgentInput) { /* 原逻辑不变 */ }
export async function* streamResume(sessionId: string, supplement: string) { /* 原逻辑不变 */
}
```

3.3 瘦身 mainGraph.ts

改造后的 `mainGraph.ts` 只剩纯图定义，大约 60 行：

```
// src/agent/graphs/mainGraph.ts
import { StateGraph, START, END, interrupt, MemorySaver } from '@langchain/langgraph'
import { HumanMessage, AIMessage } from '@langchain/core/messages'
import { z } from 'zod'
import { MainState, MainStateType } from '../state.js'
import { applySubgraph } from './applyGraph.js'
import { consultSubgraph } from './consultGraph.js'
import { createChatModel } from '../../services/llmService.js'
import { classifyPrompt, askForMorePrompt } from '../prompts.js'

const checkpointer = new MemorySaver()

async function classifyNode(state: MainStateType) {
  const allUserText = state.messages
```

```
    .filter(m => m instanceof HumanMessage)
    .map(m => m.content).join('\n')

  const reply = await createChatModel(0)
    .withStructuredOutput(z.object({ /* schema */ }))
    .invoke([new HumanMessage(classifyPrompt(allUserText))])

  return { intent: reply.intent, missingInfo: reply.missing || [], documentText:
reply.documentText || '' }
}

async function askForMoreNode(state: MainStateType) {
  const question = askForMorePrompt(state.missingInfo)
  const userAnswer = interrupt(question)
  return { messages: [new AIMessage(question), new HumanMessage(String(userAnswer))] }
}

const mainGraph = new StateGraph(MainState)
  .addNode('classify', classifyNode)
  .addNode('ask', askForMoreNode)
  .addNode('applyGraph', applySubgraph)
  .addNode('consultGraph', consultSubgraph)
  .addEdge(START, 'classify')
  .addConditionalEdges('classify', (s) => s.intent, {
    insufficient: 'ask', apply: 'applyGraph', consult: 'consultGraph'
  })
  .addEdge('ask', 'classify')
  .addEdge('applyGraph', END)
  .addEdge('consultGraph', END)

export function getCompiledGraph() {
  return mainGraph.compile({ checkpointer })
}
```

3.4 更新 routes/components/agent.ts 的 import

```
// 改前:
import { invokeAgent, resumeAgent, streamAgent, streamResume } from
'../../agent/mainGraph.js'

// 改后:
import { invokeAgent, resumeAgent, streamAgent, streamResume } from
'../../services/agentService.js'
```

3.5 验证

全部功能回归测试，确认 consult / apply / interrupt-resume / analyze / knowledge 均正常。

阶段 4：对话持久化（SqliteSaver）

> 目标：服务重启后对话不丢失

4.1 原理

当前用 `MemorySaver`，所有 checkpointer 数据存在内存里，重启就没了。
LangGraph 自带 `SqliteSaver`（@langchain/langgraph-checkpoint-sqlite），把对话状态写入 SQLite。

4.2 安装依赖

```
npm install @langchain/langgraph-checkpoint-sqlite
```

4.3 改造 mainGraph.ts

```
// 改前:
import { MemorySaver } from '@langchain/langgraph'
const checkpointer = new MemorySaver()

// 改后:
import { SqliteSaver } from '@langchain/langgraph-checkpoint-sqlite'
import path from 'path'
import { fileURLToPath } from 'url'

const __dirname = path.dirname(fileURLToPath(import.meta.url))
const CHECKPOINT_PATH = path.resolve(__dirname, '../../../data/checkpoints.db')

// SqliteSaver.fromConnString 返回 Promise，需要异步初始化
let checkpointer: SqliteSaver

export async function getCompiledGraph() {
  if (!checkpointer) {
    checkpointer = SqliteSaver.fromConnString(CHECKPOINT_PATH)
  }
  return mainGraph.compile({ checkpointer })
}
```

因为 `getCompiledGraph()` 变成了异步函数，`agentService.ts` 也要对应调整：

```
// agentService.ts
let _app: any = null

async function getApp() {
  if (!_app) _app = await getCompiledGraph()
  return _app
}

export async function invokeAgent(input: AgentInput) {
  const app = await getApp()
  const config = { configurable: { thread_id: input.sessionId } }
  const result = await app.invoke({ /* ... */ }, config)
  // ...
}
```

4.4 验证

1. 发一条消息 "挑战杯能加多少分" → agent 回复
2. 重启 `npm run dev`
3. 用同一个 sessionId 继续对话 → 应该能看到之前的对话上下文还在

阶段 5：知识库持久化（FAISS）

> 目标：服务重启后知识库不需要重新 embedding，省时间省钱

5.1 原理

当前 `MemoryVectorStore` 纯内存，每次启动都要重新 embedding 所有文件（调 API，消耗 token）。
换成 `FaissStore`，向量存在磁盘文件里，启动时直接加载。

5.2 安装依赖

```
npm install faiss-node
```

（你的 package.json 里已经有 faiss-node，确认一下就行）

5.3 改造 knowledgeManager.ts（重命名为 knowledgeService.ts 可选）

核心改动是把 `MemoryVectorStore` 换成 `FaissStore`，加上 `save()` / `load()` 逻辑：

```
import { FaissStore } from '@langchain/community/vectorstores/faiss'

const FAISS_DIR = path.resolve(__dirname, '../../data/faiss_index')

let vectorStore: FaissStore | null = null

async function getVectorStore() {
  if (!vectorStore) {
    // 尝试从磁盘加载已有索引
    if (fs.existsSync(path.join(FAISS_DIR, 'faiss.index'))) {
      vectorStore = await FaissStore.load(FAISS_DIR, createEmbeddings())
      console.log('[knowledge] 从磁盘加载 FAISS 索引')
    } else {
      vectorStore = new FaissStore(createEmbeddings(), {})
    }
  }
  return vectorStore
}

/** 每次修改后保存到磁盘 */
async function saveIndex() {
  if (vectorStore) {
    fs.mkdirSync(FAISS_DIR, { recursive: true })
    await vectorStore.save(FAISS_DIR)
```

```
  }
}
```

然后在 `ingestFile`、`removeSource` 等修改操作之后调用 `saveIndex()`。

`initKnowledge` 改为：如果 FAISS 索引已存在就直接加载（秒级），不存在才走 embedding 入库流程。

5.4 验证

1. 第一次启动：看到 `[knowledge] ✓ xxx.pdf → N 块`（正常 embedding）
2. 第二次启动：看到 `[knowledge] 从磁盘加载 FAISS 索引`（没有重新 embedding，启动快很多）
3. 上传新文件 → 确认能正常入库 + 搜索

阶段 6：简化 aiConfig

> 目标：改为 .env 为主，保留在线改配置作为可选功能

6.1 方案选择

方案 A（最简，推荐先用这个）： 删掉 DB 配置，全部走 `.env`

```
// src/services/aiConfig.ts（简化版）
export const getApiKey = () => process.env.DASHSCOPE_API_KEY ?? ''
export const getBaseUrl = () => process.env.DASHSCOPE_BASE_URL ??
'https://dashscope.aliyuncs.com/compatible-mode/v1'
export const getChatModel = () => process.env.CHAT_MODEL ?? 'qwen3-max'
export const getEmbeddingModel = () => process.env.EMBEDDING_MODEL ?? 'text-embedding-v3'
export const getSystemRole = () => process.env.SYSTEM_ROLE ?? '你是厦门大学信息学院保研加分助手。
回答语言：中文，简洁专业。'
```

对应 `.env` 文件：

```
DASHSCOPE_API_KEY=sk-xxx
DASHSCOPE_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
CHAT_MODEL=qwen3-max
EMBEDDING_MODEL=text-embedding-v3
SYSTEM_ROLE=你是厦门大学信息学院保研加分助手。...
```

同时删掉 `db/init.ts` 里的 `ai_config` 表，删掉 `routes/components/config.ts`。

方案 B（保留在线改配置）： 保持现状，不动。

建议先用方案 A，等后续确实需要在线改配置时再加回来。

6.2 验证

- 确认 `.env` 里的配置能正确读到
- 确认 agent 对话正常
- 如果用方案 A，确认管理后台的 AI 配置页面不会报错（可以在前端隐藏该入口，或返回 404）

阶段 7：增强 HITL（Human-in-the-Loop）

> 目标：在 apply 流程中增加"确认提交"步骤

7.1 当前 HITL 流程

```
用户消息 → classify → insufficient? → ask(interrupt) → 用户补充 → classify → ...
```

只在 **信息不完整** 时 interrupt。材料匹配完直接返回结果，用户没有确认环节。

7.2 升级后 HITL 流程

```
用户消息 → classify → insufficient? → ask(interrupt) → 用户补充 → classify
                                                                  ↓ apply
                                                            fetchPolicy
                                                                  ↓
                                                        analyzeAndMatch
                                                                  ↓
                                                            summarize
                                                                  ↓
                                                      confirm(interrupt) ← 【新增】
                                                                  ↓
                                                      用户确认"是/否"
                                                                  ↓ 是
                                                            submitNode ← 【新增】
```

7.3 实现步骤

在 applyGraph.ts 中加两个节点：

```
import { interrupt } from '@langchain/langgraph'

// summarize 之后新增 confirmNode
async function confirmNode(state: ApplyStateType) {
  const suggestions = state.checkResults
    .map(r => { try { return JSON.parse(r) } catch { return null } })
    .filter(Boolean)

  if (suggestions.length === 0 || suggestions[0]?.error) {
    // 没有匹配项，不需要确认
    return {}
  }

  const summary = suggestions.map((s: any) =>
    `• ${s.templateName} / ${s.ruleName}（预计 ${s.estimatedScore} 分）`
  ).join('\n')

  // interrupt 暂停，等用户确认
  const userConfirm = interrupt(
    `为您匹配到以下加分项：\n${summary}\n\n确认提交申请吗？（回复"确认"继续，其他内容取消）`
```

```
  )

  return { userConfirm: String(userConfirm) }
}

// 条件路由：用户确认 → submit，否则 → end
function confirmRouter(state: ApplyStateType): string {
  const answer = (state as any).userConfirm ?? ''
  return answer.includes('确认') ? 'submit' : 'cancel'
}

async function submitNode(state: ApplyStateType) {
  return { messages: [new AIMessage('已为您提交加分申请，请在"我的申请"中查看进度。')] }
}

async function cancelNode(state: ApplyStateType) {
  return { messages: [new AIMessage('已取消本次申请。如需重新申请，请再次发送材料信息。')] }
}
```

修改图结构：

```
export const applySubgraph = new StateGraph(ApplyState)
  .addNode('fetchPolicy', fetchPolicyNode)
  .addNode('analyzeAndMatch', analyzeAndMatchNode)
  .addNode('summarize', summarizeNode)
  .addNode('confirm', confirmNode)      // 新增
  .addNode('submit', submitNode)         // 新增
  .addNode('cancel', cancelNode)         // 新增
  .addEdge(START, 'fetchPolicy')
  .addEdge('fetchPolicy', 'analyzeAndMatch')
  .addEdge('analyzeAndMatch', 'summarize')
  .addEdge('summarize', 'confirm')       // summarize → confirm
  .addConditionalEdges('confirm', confirmRouter, {
    submit: 'submit',
    cancel: 'cancel',
  })
  .addEdge('submit', END)
  .addEdge('cancel', END)
  .compile()
```

别忘了在 state.ts 加上新字段：

```
userConfirm: Annotation<string>({ reducer: (_, x) => x, default: () => '' }),
```

7.4 验证

1. 发 "我参加了挑战杯省级一等奖，2025年9月，队长" → 走 apply 路径
2. agent 返回匹配结果 + "确认提交申请吗？" → 前端显示 interrupt UI
3. 用户输入 "确认" → 走 submit → "已为您提交加分申请"
4. 用户输入 "不了" → 走 cancel → "已取消本次申请"

各阶段执行顺序建议

顺序	阶段	难度	预计时间	依赖
1	阶段 1 - 集中管理 Prompt	★☆☆	30 分钟	无
2	阶段 2 - 统一 LLM 工厂	★☆☆	20 分钟	无
3	阶段 3 - 公共 multer + 拆分 agentService	★★☆	1 小时	阶段 1、2
4	阶段 6 - 简化 aiConfig	★☆☆	15 分钟	阶段 2
5	阶段 4 - 对话持久化	★★☆	40 分钟	阶段 3
6	阶段 5 - 知识库持久化	★★☆	40 分钟	阶段 2
7	阶段 7 - 增强 HITL	★★★	1.5 小时	阶段 3

> 阶段 1 → 2 → 3 是基础重构，先做完再做持久化和 HITL。
> 每个阶段做完都要验证，别连续做多个再测。

对前后端的影响

变更	idbackend	idfrontend / admin
阶段 1-3（内部重构）	无影响	无影响
阶段 4（SqliteSaver）	无影响	无影响
阶段 5（FAISS）	无影响	无影响
阶段 6 方案 A（删 config 路由）	调 getAIConfig/updateAIConfig 会 404	admin 的 AI 配置页面需隐藏
阶段 6 方案 B（保留）	无影响	无影响
阶段 7（HITL confirm）	无影响（interrupt 机制已有）	无影响（interrupt UI 已有）

除了阶段 6 方案 A 之外，**所有阶段都不需要改前后端。**

升级完成后的整体调用链

```
前端 → Java 后端 → Node Agent (Express)
                        │
                        ├── routes/agent.ts        解析参数
                        │       ↓
                        ├── services/agentService   invoke/stream 适配
                        │       ↓
                        ├── agent/graphs/mainGraph  图编排（classify → consult/apply/ask）
                        │       ↓                          ↓
                        │   graphs/consultGraph      graphs/applyGraph
                        │       ↓                          ↓
                        ├── agent/prompts.ts         所有 prompt 集中管理
                        ├── services/llmService      统一创建 ChatOpenAI
                        └── services/knowledgeService  FAISS 向量检索
```

每一层只做自己的事，职责清晰，改 prompt 不用翻图代码，改 LLM 配置不用改节点代码。